Binary Search Tree

A Binary Search Tree (BST) is a tree in which all the nodes follow the below-mentioned properties −

- The left sub-tree of a node has a key less than or equal to its parent node's key.
- The right sub-tree of a node has a key greater than or equal to its parent node's key.

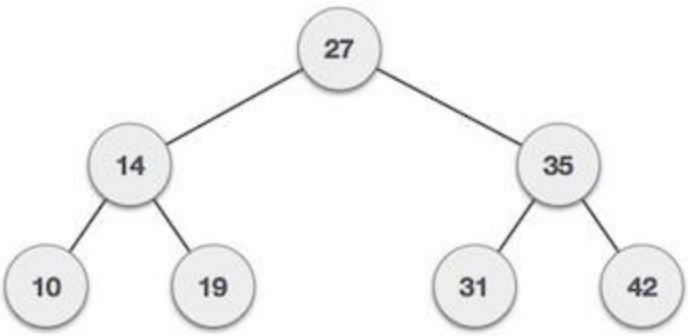
Thus, BST divides all its sub-trees into two segments; the left sub-tree and the right sub-tree and can be defined as −

```
left_subtree (keys) &leq; node (key) &leq; right_subtree (keys)
```

Binary Tree Representation

BST is a collection of nodes arranged in a way where they maintain BST properties. Each node has a key and an associated value. While searching, the desired key is compared to the keys in BST and if found, the associated value is retrieved.

Following is a pictorial representation of BST −



We observe that the root node key (27) has all less-valued keys on the left sub-tree and the higher valued keys on the right sub-tree.

Basic Operations

Following are the basic operations of a Binary Search Tree −

- **Search** − Searches an element in a tree.
- **Insert** − Inserts an element in a tree.
- **Pre-order Traversal** − Traverses a tree in a pre-order manner.
- **In-order Traversal** − Traverses a tree in an in-order manner.
- **Post-order Traversal** − Traverses a tree in a post-order manner.

Defining a Node

Define a node that stores some data, and references to its left and right child nodes.

```
struct node {
   int data;
   struct node *leftChild;
   struct node *rightChild;
};
```

Search Operation

Whenever an element is to be searched, start searching from the root node. Then if the data is less than the key value, search for the element in the left subtree. Otherwise, search for the element in the right subtree. Follow the same algorithm for each node.

Algorithm

```
1. START
2. Check whether the tree is empty or not
3. If the tree is empty, search is not possible
4. Otherwise, first search the root of the tree.
5. If the key does not match with the value in the root,
```

```
   search its subtrees.
6. If the value of the key is less than the root value,
   search the left subtree
7. If the value of the key is greater than the root value,
   search the right subtree.
8. If the key is not found in the tree, return unsuccessful search.
9. END
```

Example

Following are the implementations of this operation in various programming languages –

C | C++ | Java | **Python**

```
class Node:
   def __init__(self, data):
      self.left = None
      self.right = None
      self.data = data

# Insert method to create nodes
   def insert(self, data):
      if self.data:
         if data < self.data:
            if self.left is None:
               self.left = Node(data)
            else:
               self.left.insert(data)
         elif data > self.data:
            if self.right is None:
               self.right = Node(data)
            else:
               self.right.insert(data)
         else:
            self.data = data
# search method to compare the value with nodes
   def search(self, key):
```

```
        if key < self.data:
            if self.left is None:
                return str(key)+" Not Found"
            return self.left.search(key)
        elif key > self.data:
            if self.right is None:
                return str(key)+" Not Found"
            return self.right.search(key)
        else:
            print(str(self.data) + ' is found')

root = Node(54)
root.insert(34)
root.insert(46)
root.insert(12)
root.insert(23)
root.insert(5)
print(root.search(17))
print(root.search(12))
```

Output

```
17 Not Found
12 is found
None
```

Insertion Operation

Whenever an element is to be inserted, first locate its proper location. Start searching from the root node, then if the data is less than the key value, search for the empty location in the left subtree and insert the data. Otherwise, search for the empty location in the right subtree and insert the data.

Algorithm

```
1. START
2. If the tree is empty, insert the first element as the root node of
```

```
the
   tree. The following elements are added as the leaf nodes.
3. If an element is less than the root value, it is added into the
left
   subtree as a leaf node.
4. If an element is greater than the root value, it is added into the
right
   subtree as a leaf node.
5. The final leaf nodes of the tree point to NULL values as their
   child nodes.
6. END
```

Example

Following are the implementations of this operation in various programming languages –

C | C++ | Java | **Python**

```
class Node:
   def __init__(self, data):
      self.left = None
      self.right = None
      self.data = data

# Insert method to create nodes
   def insert(self, data):
      if self.data:
         if data  self.data:
            if self.right is None:
               self.right = Node(data)
            else:
               self.right.insert(data)
         else:
            self.data = data
   def printTree(self, prefex):
       if self is None:
           return
```

```
        self.left.printTree(prefex + "") if self.left else None
        print(prefex + "--", str(self.data),"", end = "")
        self.right.printTree(prefex + "") if self.right else None
root = Node(54)
root.insert(34)
root.insert(46)
root.insert(12)
root.insert(23)
root.insert(5)
print("Insertion Done")
print("BST: ")
root.printTree('')
```

Output

```
Insertion Done
BST:
-- 5 -- 12 -- 23 -- 34 -- 46 -- 54
```

Inorder Traversal

The inorder traversal operation in a Binary Search Tree visits all its nodes in the following order –

- Firstly, we traverse the left child of the root node/current node, if any.
- Next, traverse the current node.
- Lastly, traverse the right child of the current node, if any.

Algorithm

```
1. START
2. Traverse the left subtree, recursively
3. Then, traverse the root node
```

```
4. Traverse the right subtree, recursively.
5. END
```

Example

Following are the implementations of this operation in various programming languages –

C C++ Java **Python**

```python
class Node:
   def __init__(self, data):
      self.left = None
      self.right = None
      self.data = data

# Insert method to create nodes
   def insert(self, data):
      if self.data:
         if data < self.data:
            if self.left is None:
               self.left = Node(data)
            else:
               self.left.insert(data)
         elif data > self.data:
            if self.right is None:
               self.right = Node(data)
            else:
               self.right.insert(data)
         else:
            self.data = data

# Print the tree
   def Inorder(self):
      if self.left:
         self.left.Inorder()
         print(self.data, "->", end = " ")
      if self.right:
```

```
            self.right.Inorder()

    root = Node(54)
    root.insert(34)
    root.insert(46)
    root.insert(12)
    root.insert(23)
    root.insert(5)
    print("Inorder Traversal: ")
    root.Inorder()
```

Output

```
Inorder Traversal:
12 -> 34 -> 54 ->
```

Preorder Traversal

The preorder traversal operation in a Binary Search Tree visits all its nodes. However, the root node in it is first printed, followed by its left subtree and then its right subtree.

Algorithm

```
1. START
2. Traverse the root node first.
3. Then traverse the left subtree, recursively
4. Later, traverse the right subtree, recursively.
5. END
```

Example

Following are the implementations of this operation in various programming languages –

C C++ Java **Python**

```
class Node:
   def __init__(self, data):
      self.left = None
      self.right = None
      self.data = data
# Insert method to create nodes
   def insert(self, data):
      if self.data:
         if data < self.data:
            if self.left is None:
               self.left = Node(data)
            else:
               self.left.insert(data)
         elif data > self.data:
            if self.right is None:
               self.right = Node(data)
            else:
               self.right.insert(data)
         else:
            self.data = data

# Print the tree
   def Preorder(self):
      print(self.data, "->", end = "")
      if self.left:
         self.left.Preorder()
      if self.right:
         self.right.Preorder()
root = Node(54)
root.insert(34)
root.insert(46)
root.insert(12)
root.insert(23)
root.insert(5)
print("Preorder Traversal: ")
root.Preorder()
```

Output

```
Preorder Traversal:
54 ->34 ->12 ->5 ->23 ->46 ->
```

Postorder Traversal

Like the other traversals, postorder traversal also visits all the nodes in a Binary Search Tree and displays them. However, the left subtree is printed first, followed by the right subtree and lastly, the root node.

Algorithm

```
1. START
2. Traverse the left subtree, recursively
3. Traverse the right subtree, recursively.
4. Then, traverse the root node
5. END
```

Example

Following are the implementations of this operation in various programming languages –

C | C++ | Java | **Python**

```python
class Node:
   def __init__(self, data):
      self.left = None
      self.right = None
      self.data = data

# Insert method to create nodes
   def insert(self, data):
      if self.data:
         if data < self.data:
            if self.left is None:
               self.left = Node(data)
```

```
            else:
               self.left.insert(data)
         elif data > self.data:
            if self.right is None:
               self.right = Node(data)
            else:
               self.right.insert(data)
      else:
         self.data = data

# Print the tree
   def Postorder(self):
      if self.left:
         self.left.Postorder()
      if self.right:
         self.right.Postorder()
      print(self.data, "->", end = "")

root = Node(54)
root.insert(34)
root.insert(46)
root.insert(12)
root.insert(23)
root.insert(5)
print("Postorder Traversal: ")
root.Postorder()
```

Output

```
Postorder Traversal:
5 ->23 ->12 ->46 ->34 ->54 ->
```

Complete implementation

Following are the complete implementations of Binary Search Tree in various programming languages −

C | C++ | Java | **Python**

```
class Node:
   def __init__(self, data):
     self.left = None
     self.right = None
     self.data = data

# Insert method to create nodes
   def insert(self, data):
     if self.data:
       if data < self.data:
         if self.left is None:
            self.left = Node(data)
         else:
            self.left.insert(data)
       elif data > self.data:
         if self.right is None:
            self.right = Node(data)
         else:
            self.right.insert(data)
       else:
         self.data = data

# search method to compare the value with nodes
   def search(self, key):
     if key < self.data:
       if self.left is None:
         return str(key)+ " Not Found"
       return self.left.search(key)
     elif key > self.data:
       if self.right is None:
         return str(key)+" Not Found"
       return self.right.search(key)
     else:
       print(str(self.data) + ' is found')

# Print the tree
   def Inorder(self):
     if self.left:
```

```
        self.left.Inorder()
      print(self.data , " ->", end = " ")
      if self.right:
        self.right.Inorder()

  # Print the tree
    def Preorder(self):
      print(self.data, " ->", end = " ")
      if self.left:
        self.left.Preorder()
      if self.right:
        self.right.Preorder()

  # Print the tree
    def Postorder(self):
      if self.left:
        self.left.Postorder()
      if self.right:
        self.right.Postorder()
      print(self.data, " ->", end = " ")

  root = Node(54)
  root.insert(34)
  root.insert(46)
  root.insert(12)
  root.insert(23)
  root.insert(5)
  print("Insertion Done")
  print("Preorder Traversal: ")
  root.Preorder()
  print("\nInorder Traversal: ")
  root.Inorder()
  print("\nPostorder Traversal: ")
  root.Postorder()
  ele = 17
  print("\nElement to be searched: ", ele)
  print(root.search(ele))
```

Output

```
Insertion Done
Preorder Traversal:
54  -> 34  -> 12  -> 5  -> 23  -> 46  ->
Inorder Traversal:
5  -> 12  -> 23  -> 34  -> 46  -> 54  ->
Postorder Traversal:
5  -> 23  -> 12  -> 46  -> 34  -> 54  ->
Element to be searched:  17
17 Not Found
```